Retrieval-Augmented Generation (RAG) for Corporate Wikis/Documentation

Authors: Yasmine Huo, Tracy Ling
Team: Insight Makers

Progess done between Mid-semester presentation

1. Finalize preprocessing pipeline
2. implement a basic retrieval-augmented question answering pipeline

Preprocessing Pipeline Overview

We used a **three-pass filtering pipeline**:

1. **Pass 1 (broad recall-first filter)**
   Use a glossary-based keyword scoring rule to keep potentially relevant CS articles.

2. **Review 1**
   Randomly sample 100 articles and inspect top matched terms to identify false positives.

3. **Pass 2 (refined high-confidence filter)**
   Replace the broad glossary with high-confidence CS terms and stricter keep rules.

4. **Review 2**
   Again inspect random samples and matched-term distributions.

5. **Pass 3 (precision-focused filter)**
   Keep only core CS evidence, add support terms, block weak/ambiguous terms, and switch to whole-word matching.

6. **Chunk building**
   Convert the final filtered article set into structured chunks with metadata and section-aware paths.

File map used in this report

- `filter_articles.py` → Pass 1
- `review_filtered_articles.py` → Review 1
- `filter_articles_second_pass.py` → Pass 2

- `review_filtered_articles_v2.py` → Review 2
- `filter_articles_third_pass.py` → Pass 3
- `review_filtered_articles_v3.py` → Review 3
- `build_chunks_from_filtered_articles.py` → final chunk construction

Step 1 — First-pass filtering (`filter_articles.py`)

Objective

The first pass was intentionally **broad**.
The goal was to capture as many potentially CS-related pages as possible from the WikiExtractor output.

Main idea

We used a glossary of CS-related terms and scored each article based on **where the terms appeared**:

- title match → strongest signal
- first two paragraphs → medium signal
- later content → weaker signal

This first pass acted as a coarse filter to create a candidate pool.

In [1]:
```
# Pass 1: scoring and keep rules
def normalize_text(text):
    if not text:
        return ""
    text = text.lower()
    text = re.sub(r"\s+", " ", text).strip()
    return text


def split_paragraphs(text):
    return [p.strip() for p in text.split("\n") if p.strip()]


def score_article(title, text, glossary_terms):
    title_l = normalize_text(title)
    paragraphs = split_paragraphs(text)

    first_part = normalize_text(" ".join(paragraphs[:2]))
    rest_part = normalize_text(" ".join(paragraphs[2:]))

    score = 0
```

```
    matched_terms = set()

    for term in glossary_terms:
        if term in title_l:
            score += 3
            matched_terms.add(term)

        if term in first_part:
            score += 2
            matched_terms.add(term)
        elif term in rest_part:
            score += 1
            matched_terms.add(term)

    return score, matched_terms


def keep_article(score, matched_terms):
    return (score >= 4) or (len(matched_terms) >= 2)
```

Scoring Rule:

score glossary-term matches by location:

- title = +3
- first part = +2
- rest = +1

An article is kept if:

- `score >= 4`, **or**
- it matches at least **two distinct glossary terms**

Review 1 — Inspecting first-pass results

Why we reviewed the results

After the first filter, we needed to check whether the broad glossary was producing too many false positives.

We used two validation strategies:

A. Random 100 samples inspection

A review script generated a random sample of retained articles so we could manually label them as:

- `cs`

- `related`
- `non_cs`

B. Top matched terms

We also counted the most frequent matched terms across the retained set.
This helped us identify terms that looked “technical” but were actually **too generic** and caused many false positives.

In [ ]:
```
# Review 1 sample output
sample_text = r'''[1]
ID: 52118099
Title: Blair Branch
Score: 6
Matched terms: ['set', 'state', 'stream']
URL: https://en.wikipedia.org/wiki?curid=52118099
Preview:
Blair Branch is a stream in Lewis County in the U.S. state of Missouri. It i
Blair Branch has the name of Henry Blair, an early settler.

Manual label: ____________________
Notes: ___________________________

==============================================================================

[2]
ID: 2459533
Title: Valley Yokuts
Score: 3
Matched terms: ['class', 'ide']
URL: https://en.wikipedia.org/wiki?curid=2459533
Preview:
Valley Yokuts is a dialect cluster of the Yokuts language of California.
Chukchansi, which is still spoken natively, has language classes and a presc
Varieties.
Valley Yokuts is sometimes considered three languages.
Of these, "Yawelmani" , also known as "Yowlumni", is the best known. See als

Manual label: ____________________
Notes: ___________________________

==============================================================================

[3]
ID: 263659
Title: A20 line
Score: 25
Matched terms: ['bit', 'byte', 'compiler', 'computer', 'data', 'event', 'exe
URL: https://en.wikipedia.org/wiki?curid=263659
Preview:
The A20, or address line 20, is one of the electrical lines that make up the
A microprocessor typically has a number of address lines equal to the base-t
```

```
Manual label: _____________________
Notes: ___________________________

================================================================================

[4]
ID: 64716982
Title: Wide Open Walls (Sacramento Mural Festival)
Score: 13
Matched terms: ['class', 'event', 'ide', 'list', 'state', 'tree']
URL: https://en.wikipedia.org/wiki?curid=64716982
Preview:
Wide Open Walls (Sacramento Mural Festival) is an annual street art event he
Founders established the interactive art event to engage both Sacramento res

Manual label: _____________________
Notes: ___________________________

================================================================================

[5]
ID: 12575646
Title: Superfantozzi
Score: 8
Matched terms: ['bit', 'character', 'ide', 'inheritance', 'list', 'memory',
URL: https://en.wikipedia.org/wiki?curid=12575646
Preview:
Superfantozzi is an Italian film from 1986. It is the fifth film in the saga
```

```python
print(sample_text[:3500])
```

In [ ]:

```python
# Review 1 top matched terms (truncated)
top_terms_v1 = r'''Top matched terms in wiki_cs_articles.jsonl
==================================================

ide     2808478
state   1717006
list    1347568
set     1240289
event   1075355
record  858806
bit     766339
class   738998
field   685116
tree    532018
character       478683
collection      267838
reference       263519
method  224860
stream  219651
object  204166
data    190094
sequence        153682
statement       127993
computer        124734
user    109913
handle  109026
```

```
memory  90707
closure 85810
expression      81896
string  80928
internet        78150
software        69586
procedure       67108
server  65830
storage 62051
domain  55576
client  54581
execution       50732
heap    45146
variable        43643
declaration     43071
download        42426
byte    34949
database        34502
interface       29298
artifact        28525
java    25547
stack   25391
parameter       25049
documentation   24596
computing       22914
algorithm       21924
node    21915
container       19994
computation     19875
inheritance     19740
coding  19534
matrix  19237
iteration       18499
upload  17639
conditional     17019
methodology     15667
operating system        13474
interpreter     13187
peripheral      13162
artificial intelligence 12872
assertion       12746
intellectual property   9623
integer 8964
cipher  8105
benchmark       8060
abstraction     7969
programming language    7080
pointer 6955
computer program        6898
prolog  6858
syntax  6776
kernel  6699
python  6623
robotics        6071
compiler        5886
identifier      5842
```

```
user interface  5821
bandwidth       5637
computer scientist      5525
source code     5152
queue   5139
machine learning        5071
blacklist       4768
invariant       4537
wi-fi   4483
concurrency     4170
software development    4160
loader  4005
encryption      3845
semantics       3502
feasibility study       3162
modem   2884
data center     2825
computer graphics       2824
serialization   2801
linker  2772
computer network        2724
ascii   2713
cryptography    2437
bioinformatics  2238
router  2174
precondition    2131
number theory   2124
data structure  2120
software engineering    1988
computer progr'''
print(top_terms_v1[:1800])
```

Conclusion from Review 1

Pass 1 succeeded as a candidate generator, but it was **not precise enough**.
So the next step was to move from a broad glossary to a more curated, high-confidence term list.

Step 2 — Second-pass filtering (`filter_articles_second_pass.py`)

Objective

The second pass aimed to **reduce false positives** while keeping genuinely CS-related pages.

Key change

Instead of using a very broad glossary, we manually defined:

- **HIGH_CONFIDENCE_TERMS**
  strong CS indicators
- **TITLE_ONLY_TERMS**
  weaker but still helpful terms, only trusted in article titles

This was an important shift from a recall-first strategy to a more balanced precision/recall tradeoff.

In [ ]:
```
# Pass 2: curated high-confidence CS terms
HIGH_CONFIDENCE_TERMS = sorted(set([
    "algorithm",
    "algorithm design",
    "application programming interface",
    "application software",
    "artificial intelligence",
    "ascii",
    "automata theory",
    "bandwidth",
    "benchmark",
    "binary number",
    "bioinformatics",
    "bit rate",
    "booting",
    "boolean algebra",
    "callback",
    "central processing unit",
    "cipher",
    "cloud computing",
    "coding theory",
    "compiler",
    "computation",
    "computability theory",
    "computational biology",
    "computational chemistry",
    "computational complexity theory",
    "computational model",
    "computational neuroscience",
    "computational physics",
    "computational science",
    "computer architecture",
    "computer graphics",
    "computer network",
    "computer programming",
    "computer program",
    "computer science",
    "computer scientist",
    "computer security",
    "computer vision",
    "computing",
    "concurrency",
    "control flow",
    "creative commons",
```

```
    "cryptography",
    "csv",
    "cyberspace",
    "data center",
    "data mining",
    "data science",
    "data structure",
    "data type",
    "database",
    "daemon",
    "debugging",
    "digital data",
    "digital signal processing",
    "distributed computing",
    "dns",
    "domain name system",
    "emulator",
    "encryption",
    "executable",
    "exception handling",
    "feasibility study",
    "filename extension",
    "floating-point arithmetic",
    "for loop",
    "formal methods",
    "functional programming",
    "game theory",
    "gigabyte",
    "graph theory",
    "hash function",
    "hash table",
    "human-computer interaction",
    "image processing",
    "information retrieval",
    "input/output",
    "integrated development environment",
    "intelligent agent",
    "interface",
    "interpreter",
    "iteration",
    "java",
    "kernel",
    "linker",
    "linked list",
    "loader",
    "logic programming",
    "machine learning",
    "machine vision",
    "mathematical logic",
    "matrix",
    "modem",
    "natural language processing",
    "node",
    "number theory",
    "numerical analysis",
    "numerical method",
```

```
    "object code",
    "object-oriented programming",
    "open-source software",
    "operating system",
    "optical fiber",
    "parallel computing",
    "parameter",
    "peripheral",
    "pointer",
    "programming language",
    "prolog",
    "python",
    "quantum computing",
    "queue",
    "r programming language",
    "radix",
    "recursion",
    "relational database",
    "robotics",
    "router",
    "run time",
    "search algorithm",
    "semantics",
    "serialization",
    "software",
    "software design",
    "software development",
    "software engineering",
    "software testing",
    "source code",
    "stack",
    "subroutine",
    "syntax",
    "technical documentation",
    "type theory",
    "user agent",
    "user interface",
    "user interface design",
    "variable",
    "virtual machine",
    "wi-fi",
    "xhtml",
]))


# ==============================
# Optional weaker-but-still-useful terms
# Only count in title, not body
# ==============================
TITLE_ONLY_TERMS = sorted(set([
    "computer",
    "computing",
    "database",
    "software",
    "compiler",
    "kernel",
```

```
    "algorithm",
    "programming language",
    "machine learning",
    "artificial intelligence",
    "computer graphics",
    "computer network",
    "computer vision",
    "data structure",
    "information retrieval",
    "operating system",
    "cryptography",
    "computer architecture",
]))


def normalize_text(text: str) -> str:
    if not text:
        return ""
    text = text.lower()
    text = re.sub(r"\s+", " ", text).strip()
    return text


def split_paragraphs(text: str) -> list[str]:
    return [p.strip() for p in text.split("\n") if p.strip()]


def count_matches(text: str, terms: list[str]) -> set[str]:
    matched = set()
    for term in terms:
        if term in text:
            matched.add(term)
    return matched


def score_article(title: str, text: str):
    title_l = normalize_text(title)
    paragraphs = split_paragraphs(text)

    first_part = normalize_text(" ".join(paragraphs[:2]))
    rest_part = normalize_text(" ".join(paragraphs[2:]))

    score = 0
    matched_terms = set()

    # Strong signals
    title_matches = count_matches(title_l, HIGH_CONFIDENCE_TERMS)
    first_matches = count_matches(first_part, HIGH_CONFIDENCE_TERMS)
    rest_matches = count_matches(rest_part, HIGH_CONFIDENCE_TERMS)

    # Weaker title-only signals
    title_only_matches = count_matches(title_l, TITLE_ONLY_TERMS)

    # Scoring
    score += 5 * len(title_matches)
    score += 3 * len(first_matches)
```

```
    score += 1 * len(rest_matches)
    score += 1 * len(title_only_matches - title_matches)

    matched_terms.update(title_matches)
    matched_terms.update(first_matches)
    matched_terms.update(rest_matches)
    matched_terms.update(title_only_matches)

    return score, matched_terms, {
        "title_matches": sorted(title_matches),
        "first_matches": sorted(first_matches),
        "rest_matches": sorted(rest_matches),
        "title_only_matches": sorted(title_only_matches),
    }


def keep_article(score: int, details: dict) -> bool:
    title_hits = len(details["title_matches"])
    first_hits = len(details["first_matches"])
    total_strong_hits = title_hits + first_hits + len(details["rest_matches"

    # Strict rules:
    # 1) strong title hit
    if title_hits >= 1:
        return True

    # 2) at least two strong hits in first paragraphs
    if first_hits >= 2:
        return True

    # 3) enough overall strong evidence
    if total_strong_hits >= 3 and score >= 6:
        return True

    return False


def main():
    total_articles = 0
    kept_articles = 0

    OUTPUT_FILE.parent.mkdir(parents=True, exist_ok=True)

    with open(INPUT_FILE, "r", encoding="utf-8") as fin, \
         open(OUTPUT_FILE, "w", encoding="utf-8") as fout:

        for line in fin:
            total_articles += 1
            article = json.loads(line)

            title = article.get("title", "")
            text = article.get("text", "")

            if not text.strip():
                continue
```

```
            score, matched_terms, details = score_article(title, text)

            if keep_article(score, details):
                article["cs_filter_score_v2"] = score
                article["matched_terms_v2"] = sorted(matched_terms)[:50]
                article["title_matches_v2"] = details["title_matches"]
                article["first_matches_v2"] = details["first_matches"]
                article["rest_matches_v2"] = details["rest_matches"]
                fout.write(json.dumps(article, ensure_ascii=False) + "\n")
                kept_articles += 1

            if total_articles % 100000 == 0:
                print(f"Processed {total_articles:,} articles... kept {kept_

    print(f"Total articles seen: {total_articles:,}")
    print(f"Kept CS articles v2: {kept_articles:,}")
    print(f"Saved to: {OUTPUT_FILE}")


if __name__ == "__main__":
    main()
```

How Pass 2 improved the logic

Pass 2 still used position-aware scoring, but the signals were more selective.

Evidence weighting

- strong term in title → very strong signal
- strong term in first two paragraphs → strong signal
- strong term later in article → weaker support
- weak term in title only → small bonus

Keep rule

An article is kept if it satisfies one of the following:

1. at least one strong title match,
2. at least two strong matches in the first two paragraphs,
3. enough combined strong evidence with a score threshold.

This made Pass 2 much stricter than Pass 1.

Review 2 — Evaluating the second-pass results

So we repeated the same validation process:

1. random sample review,
2. top matched term inspection.

In [ ]:
```
# Review 2 sample output (truncated)
sample_text_v2 = r'''[1]
ID: 4853616
Title: Replay Professional
Score v2: 6
Title matches v2: []
First matches v2: ['interface', 'software']
Rest matches v2: []
Matched terms v2: ['interface', 'software']
URL: https://en.wikipedia.org/wiki?curid=4853616
Preview:
Replay Professional was a sound sampling product for the Atari ST. This was
It consisted of a cartridge which interfaced an analog to digital converter
It included a suite of offline DSP functions Fast Fourier transform, a range
Compact discs were a relatively new consumer product at that time, and the f

Manual label: ____________________
Notes: ___________________________

==============================================================================

[2]
ID: 9852982
Title: Syndie
Score v2: 6
Title matches v2: []
First matches v2: ['computer network']
Rest matches v2: ['interface', 'java', 'user interface']
Matched terms v2: ['computer network', 'interface', 'java', 'user interface'
URL: https://en.wikipedia.org/wiki?curid=9852982
Preview:
Syndie is an open-source cross-platform computer application to syndicate (r
Syndie is capable of reaching archives situated in the following anonymous r
History.
Syndie has been in development since 2003 and ties in closely with the I2P r
Following the departure of lead developer Jrandom in 2007, work on syndie

Manual label: ____________________
Notes: ___________________________

==============================================================================

[3]
ID: 21164918
Title: Touch Typist Typing Tutor
Score v2: 6
Title matches v2: []
First matches v2: ['software']
Rest matches v2: ['compiler', 'operating system', 'software']
Matched terms v2: ['compiler', 'operating system', 'software']
URL: https://en.wikipedia.org/wiki?curid=21164918
Preview:
```

```
Touch Typist Typing Tutor is developed by Sector Software.
Touch Typist typing tutor is the earliest example of typing tutor software c

Manual label: _____
Notes: _____

==============================================================================

[4]
ID: 30806403
Title: List of concurrent and parallel programming languages
Score v2: 19
Title matches v2: ['programming language']
First matches v2: ['concurrency', 'executable', 'programming language', 'syr
Rest matches v2: ['application programming interface', 'interface']
Matched terms v2: ['application programming interface', 'concurrency', 'exec
print(sample_text_v2[:3200])
```

In [9]:

```
# Review 2 top matched terms (truncated)
top_terms_v2 = r'''Top matched terms in wiki_cs_articles_v2.jsonl
==================================================

software        27305
interface       13678
computing       12932
algorithm       12733
computation     10596
operating system        9057
computer science        8481
database        7997
variable        7589
parameter       6700
programming language    6323
node    6269
java    6197
artificial intelligence 4936
user interface  4702
matrix  4456
stack   4019
source code     4002
computer scientist      3986
computer program        3425
machine learning        3410
kernel  3328
compiler        3163
software development    3037
syntax  2911
python  2691
bandwidth       2365
iteration       2344
encryption      2339
peripheral      1887
data structure  1879
robotics        1830
semantics       1813
open-source software    1598
```

```
pointer 1596
cryptography    1596
benchmark       1584
executable      1522
software engineering    1500
computer network        1443
interpreter     1392
virtual machine 1330
cipher  1328
computer        1310
router  1306
computer programming    1283
modem   1270
ascii   1253
wi-fi   1241
queue   1223
cloud computing 1218
data type       1194
computer graphics       1184
bioinformatics  1109
data center     1105
emulator        1080
computer vision 1072
debugging       1005
image processing        923
relational database     894
natural language processing     877
data science    872
application programming interface       846
r programming language  819
loader  814
input/output    798
dns     797
software design 787
graph theory    781
subroutine      753
data mining     738
computer security       712
distributed computing   668
object-oriented programming     656
recursion       648
computer architecture   630
application software    596
computational model     593
number theory   580
quantum computing       570
integrated development environment      559
prolog  548
numerical analysis      545
computational biology   539
daemon  525
concurrency     498
information retrieval   494
booting 491
gigabyte        464
functional programming  460
```

```
central processing unit 459
game theory'''
print(top_terms_v2[:1800])
```

```
Top matched terms in wiki_cs_articles_v2.jsonl
==================================================

software        27305
interface       13678
computing       12932
algorithm       12733
computation     10596
operating system        9057
computer science        8481
database        7997
variable        7589
parameter       6700
programming language    6323
node    6269
java    6197
artificial intelligence 4936
user interface  4702
matrix  4456
stack   4019
source code     4002
computer scientist      3986
computer program        3425
machine learning        3410
kernel  3328
compiler        3163
software development    3037
syntax  2911
python  2691
bandwidth       2365
iteration       2344
encryption      2339
peripheral      1887
data structure  1879
robotics        1830
semantics       1813
open-source software    1598
pointer 1596
cryptography    1596
benchmark       1584
executable      1522
software engineering    1500
computer network        1443
interpreter     1392
virtual machine 1330
cipher  1328
computer        1310
router  1306
computer programming    1283
modem   1270
ascii   1253
wi-fi   1241
queue   1223
cloud computing 1218
data type       1194
computer graphics       1184
```

```
bioinformatics  1109
data center     1105
emulator        1080
computer vision 1072
debugging       1005
image processing        923
relational database     894
natural language processing     877
data science    872
application programming interface       846
r programming language  819
loader  814
input/output    798
dns     797
software design 787
graph theory    781
subroutine      753
data mining     738
computer security       712
distributed computing   668
object-oriented programming     656
recursion       648
computer architecture   630
application software    596
computational model     593
number theory   580
quantum computing       570
integrated development environment      559
prolog  548
numerical analysis      545
computational biology   539
daemon  525
concurrency     498
information retrieval   494
booting 491
gigabyte        464
functional programming  460
central processing unit 459
game theory
```

What we learned from Review 2

Pass 2 was **much better** than Pass 1.
More retained pages now looked like genuine technical or computing articles.

However, some issues still remained.

Remaining ambiguity

A few terms were still too broad or ambiguous, such as:

- `java`
- `node`

- `variable`
- `software`
- `interface`
- `computing`

These terms can be relevant to CS, but by themselves they are not always reliable enough to decide article relevance.

Step 3 — Third-pass filtering (`filter_articles_third_pass.py`)

Objective

The third pass focused on **precision**.

This step was designed to keep only articles with strong evidence that they are genuinely about computer science or closely related technical topics.

Main improvements

Compared with Pass 2, Pass 3 introduced three important refinements:

1. **Core vs. support terms**
   - core terms drive the decision,
   - support terms only help slightly.
2. **Blocked / weak terms**
   - ambiguous items like `java`, `node`, and `variable` are no longer trusted as standalone evidence.
3. **Whole-word matching**
   - we replaced loose substring matching with exact word-boundary matching.

In [10]:
```
# Pass 3: core/support/weak-term design
CORE_TERMS = [
    "computer science",
    "programming language",
    "operating system",
    "database",
    "compiler",
    "cryptography",
    "computer security",
    "machine learning",
    "artificial intelligence",
```

```
    "computer network",
    "software engineering",
    "data structure",
    "algorithm",
    "computer programming",
    "computer program",
    "source code",
    "formal methods",
    "distributed computing",
    "information retrieval",
    "computer vision",
    "computer graphics",
]

SUPPORT_TERMS = [
    "software",
    "interface",
    "computing",
    "open-source software",
    "user interface",
    "technical documentation",
    "operating system",
    "kernel",
    "virtual machine",
    "programming language",
    "database",
    "computer science",
]

BLOCK_OR_WEAK_TERMS = {
    "java",
    "node",
    "prolog",
    "variable",
    "ascii",
}
```

In [ ]:

```
# Pass 3: whole-word matching and score logic
def normalize_text(text: str) -> str:
    if not text:
        return ""
    text = text.lower()
    text = re.sub(r"\s+", " ", text).strip()
    return text

def split_paragraphs(text: str):
    return [p.strip() for p in text.split("\n") if p.strip()]

def contains_term_whole_word(text: str, term: str) -> bool:
    pattern = r"\b" + re.escape(term.lower()) + r"\b"
    return re.search(pattern, text) is not None

def matched_terms(text: str, terms: list[str]) -> list[str]:
    hits = []
    for term in terms:
        if contains_term_whole_word(text, term):
```

```
            hits.append(term)
    return hits

def score_article(title: str, text: str):
    title_l = normalize_text(title)
    paragraphs = split_paragraphs(text)
    first_part = normalize_text(" ".join(paragraphs[:2]))
    rest_part = normalize_text(" ".join(paragraphs[2:]))

    title_core = matched_terms(title_l, CORE_TERMS)
    first_core = matched_terms(first_part, CORE_TERMS)
    rest_core = matched_terms(rest_part, CORE_TERMS)

    title_support = matched_terms(title_l, SUPPORT_TERMS)
    first_support = matched_terms(first_part, SUPPORT_TERMS)
    rest_support = matched_terms(rest_part, SUPPORT_TERMS)

    title_core = [t for t in title_core if t not in BLOCK_OR_WEAK_TERMS]
    first_core = [t for t in first_core if t not in BLOCK_OR_WEAK_TERMS]
    rest_core = [t for t in rest_core if t not in BLOCK_OR_WEAK_TERMS]

    score = 0
    score += 5 * len(set(title_core))
    score += 3 * len(set(first_core))
    score += 1 * len(set(rest_core))
    score += 1 * len(set(title_support))
    score += 1 * len(set(first_support))

    details = {
        "title_core": sorted(set(title_core)),
        "first_core": sorted(set(first_core)),
        "rest_core": sorted(set(rest_core)),
        "title_support": sorted(set(title_support)),
        "first_support": sorted(set(first_support)),
    }
    return score, details

def keep_article(score: int, details: dict) -> bool:
    if len(details["title_core"]) >= 1:
        return True
    if len(details["first_core"]) >= 2:
        return True
    if (len(details["title_core"]) + len(details["first_core"]) + len(detail
        return True
    return False
```

Keep rule in Pass 3

The final rule is intentionally simple:

- keep if the title contains a core term, or
- keep if the first two paragraphs contain strong core evidence, or
- keep if total core evidence plus score is strong enough.

At this point, support terms can help, but they do **not** dominate the decision.

Review 3 — Final inspection before chunking

Again, we used:

1. random sample inspection,
2. top matched core-term counts by position.

In [ ]:
```
# Review 3 sample output (truncated)
sample_text_v3 = r'''[1]
ID: 19981418
Title: ORCA/Modula-2
Score v3: 10
Title core v3: []
First core v3: ['compiler', 'programming language']
Rest core v3: ['compiler', 'operating system', 'source code']
URL: https://en.wikipedia.org/wiki?curid=19981418
Preview:
ORCA/Modula-2 is a Modula-2 compiler written in the Modula-2 programming lar
It was developed by Peter Easdown during 1993–94 and published by The Byte W
The compiler was originally developed using a version of TopSpeed Modula-2 c

Manual label: _____
Notes: _____

==============================================================================

[2]
ID: 34720697
Title: Lists of database management systems
Score v3: 11
Title core v3: ['database']
First core v3: ['database']
Rest core v3: ['database']
URL: https://en.wikipedia.org/wiki?curid=34720697
Preview:
Lists of database management systems provide indexes and/or comparisons of c
They include:
See for a complete lists of articles about database management systems.

Manual label: _____
Notes: _____

==============================================================================

[3]
ID: 16234982
Title: 3D reconstruction
Score v3: 10
Title core v3: []
First core v3: ['computer graphics', 'computer vision']
```

```
Rest core v3: ['algorithm', 'computer graphics', 'computer vision', 'machine
URL: https://en.wikipedia.org/wiki?curid=16234982
Preview:
In computer vision and computer graphics, 3D reconstruction is the process c
This process can be accomplished either by active or passive methods. If the
Motivation and applications.
The research of 3D reconstruction has always been a difficult goal. By Using

Manual label: ____________________
Notes: ___________________________

==============================================================================

[4]
ID: 47087
Title: AIBO
Score v3: 8
Title core v3: []
First core v3: ['artificial intelligence']
Rest core v3: ['algorithm', 'artificial intelligence', 'computer science', '
URL: https://en.wikipedia.org/wiki?curid=47087
Preview:
AIBO (Artificial Intelligence RoBOt, homonymous with , "pal" or "partner" ir

Manual label: _________'''
print(sample_text_v3[:3200])
```

In [ ]:

```
# Review 3 top matched core terms (truncated)
top_terms_v3 = r'''Top matched core terms in wiki_cs_articles_v3.jsonl
============================================================

first::computer science 3125
rest::algorithm 2548
rest::computer science  2507
rest::database  2147
rest::operating system  1977
first::database 1628
first::programming language     1590
rest::programming language      1576
first::algorithm        1555
first::operating system 1551
rest::artificial intelligence   1518
rest::source code       1442
first::artificial intelligence  1395
rest::compiler  1314
rest::machine learning  1183
first::machine learning 904
title::algorithm        646
first::source code      609
title::database 550
first::compiler 543
rest::software engineering      507
rest::data structure    500
first::cryptography     458
rest::computer vision   430
rest::cryptography      420
```

```
first::computer programming     404
first::software engineering     395
title::programming language     376
first::computer graphics        317
first::computer vision  310
first::computer program 300
rest::computer graphics 295
rest::computer programming      263
rest::computer program  258
first::data structure   254
rest::distributed computing     240
title::computer science 224
first::computer security        205
first::distributed computing    189
title::artificial intelligence  183
rest::information retrieval     170
rest::computer security 170
title::operating system 154
first::information retrieval    132
rest::formal methods    124
first::computer network 102
rest::computer network  100
title::cryptography     98
first::formal methods   66
title::software engineering     51
title::computer graphics        50
title::compiler 49
title::computer programming     48
title::machine learning 48
title::computer security        34
title::data structure   25
title::information retrieval    25
title::computer vision  22
title::source code      19
title::computer program 18
title::distributed computing    17
title::compute'''
print(top_terms_v3[:1800])
```

What Review 3 showed

At this stage, the retained pages were much more concentrated around:

- computer science,
- programming languages,
- databases,
- operating systems,
- machine learning,
- algorithms,
- source code,
- computer graphics / vision / security / IR.

There could still be some borderline cases, but overall the retained corpus was much cleaner and more suitable for downstream chunking and retrieval.

Evaluation Metrics for the Three Filtering Stages

To evaluate the quality of the article filtering pipeline, we manually reviewed 100 randomly sampled articles from each stage and assigned each article to one of three categories:

- **cs**: clearly belongs to core computer science content
- **related**: not purely core CS, but still clearly related to CS / computing / software / AI
- **non_cs**: clearly unrelated and should not be kept

Based on these labels, we computed the following metrics.

Metric Definitions

- **strict precision**
  The proportion of sampled articles that are clearly core CS.

  $$\text{strict precision} = \frac{\#cs}{100}$$

- **relaxed precision**
  The proportion of sampled articles that are acceptable under our broader goal of keeping all CS-related content.

  $$\text{relaxed precision} = \frac{\#cs + \#related}{100}$$

- **false positive rate**
  The proportion of sampled articles that are clearly not CS-related and should not have been retained.

  $$\text{false positive rate} = \frac{\#non_cs}{100}$$

- **related rate**
  The proportion of sampled articles that are not core CS, but are still reasonably related to CS.

  $$\text{related rate} = \frac{\#related}{100}$$

Results

Review Stage	cs	related	non_cs	strict precision	relaxed precision	fals positiv rat
First-round review (`wiki_cs_sample_for_review`)	1	1	98	1%	2%	98
Second-round review (`wiki_cs_v2_sample_for_review`)	42	33	25	42%	75%	25
Third-round review (`wiki_cs_v3_sample_for_review`)	62	37	1	62%	99%	1

Second Progress

These two scripts implement a **basic retrieval-augmented question answering pipeline**:

1. `vector.py`
    - reads chunked CS-related Wikipedia data from `corpus_chunks_filtered.jsonl`
    - converts each chunk into a LangChain `Document`
    - embeds the documents with an embedding model
    - stores them in a Chroma vector database
    - creates a retriever for semantic search
2. `answer.py`
    - imports the retriever created in `vector.py`
    - loads an LLM (`qwen2.5:3b`) through Ollama
    - retrieves the top relevant chunks for a user question
    - injects those chunks into a prompt
    - asks the model to answer using only the retrieved context

So together, the two files form a simple end-to-end RAG system:

build vector store -> retrieve relevant chunks -> generate answer from retrieved context

Running the Chatbot in Terminal:
cd project-rag
source .venv/bin/activate
python3 -m src.generation.answer

GitHub Link

https://github.com/Washu-Project-Rag/project-rag